

UNIVERSIDADE ESTADUAL DE MONTES CLAROS – UNIMONTES
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS – CCET
DEPARTAMENTO DE CIÊNCIAS DA COMPUTAÇÃO – DCC

EXERCÍCIO - CAPÍTULO III

RAMON LOPES DE QUEIROZ

MONTES CLAROS – MG
AGOSTO/2026

RAMON LOPES DE QUEIROZ

EXERCÍCIO - CAPÍTULO III

Atividade avaliativa apresentada para atendimento de requisito parcial para aprovação na disciplina Engenharia de Software do Curso de Graduação em Bacharelado em Sistemas de Informação – 3º período

Professor: Dra. Marilee Patta

MONTES CLAROS – MG

AGOSTO/2026

Capítulo 3

EXERCÍCIOS

1 Para as empresas, a entrega rápida e implantação de novos sistemas frequentemente são mais importantes do que a funcionalidade detalhada desses sistemas. Por que?

2 Como os princípios básicos dos métodos ágeis levam ao desenvolvimento e implantação de software acelerados.

3 Quando não se recomenda o uso de um método ágil para o desenvolvimento de um sistema de software?

4 Extreme Programming expressa os requisitos dos usuários como Estória, sendo cada uma escrita em um cartão. Discuta as vantagens e desvantagens dessa abordagem para a descrição de requisitos.

5 Explique por que o desenvolvimento test-first ajuda o programador a desenvolver um melhor entendimento dos requisitos do sistema. Quais são as potenciais dificuldades com o desenvolvimento test-first?

6 Mostre quatro razões pelas quais a taxa de produtividade de programadores que trabalham em pares pode ser mais que a metade da taxa de produtividade de dois programadores que trabalham individualmente.

7 Compare e contraste a abordagem xp e Scrum

8 Um gerente de software desenvolve softwares críticos de controles para aeronaves. O responsável pelo desenvolvimento do sistema de apoio ao projeto de software dá suporte para a tradução de requisitos de software em uma especificação formal de software. Quais as vantagens e desvantagens das estratégias de desenvolvimento a seguir:

a) Coletar dos engenheiros de software e stakeholders externos (como a autoridade regulatória de certificação) os requisitos para um sistema desse tipo e desenvolver o sistema usando uma abordagem dirigida a planos.

b) Desenvolver um protótipo usando uma linguagem de script, como Ruby ou Python, avaliar esse protótipo com os engenheiros de software e outros stakeholders e, em seguida, revisar os requisitos do sistema. Desenvolver novamente o sistema final, usando Java.

c) Desenvolver o sistema em Java, usando uma abordagem ágil com um usuário envolvido na equipe de desenvolvimento.

3.9 Um dos problemas de se ter usuários participando de uma equipe de desenvolvimento de software é que eles 'se tornam nativos', ou seja, adotam a perspectiva da equipe de desenvolvimento e perdem de vista as necessidades de seus

colegas usuários. Sugira três maneiras de evitar esse problema e discuta as vantagens e desvantagens de cada abordagem.

10 Para reduzir os custos e o impacto ambiental das viagens, a empresa decide fechar uma série de escritórios e dar suporte ao pessoal para trabalhar em casa. No entanto, a gerência sênior que introduz essa política não está ciente de que o software é desenvolvido por métodos ágeis, que contam com equipe trabalhando no mesmo local, e a programação em pares. Discuta as dificuldades que essa nova política pode causar e como você poderia contornar esses problemas.

Exercício - Capítulo 3, Engenharia de Software:

1 - Para as empresas, é mais vantajoso esse método, por conta de alguns fatores: A obsolescência, ou seja, utilizando de um produto que logo ficará ultrapassado, a velocidade que a tecnologia evolui e torna diversas operações mais rápidas e simples, recuperar o investimento mais rápido, concorrência de mercado, entre outros.

2 - Existem alguns fatores do método ágil que acabam agilizando o processo, em comprometimento da sua robustez, como por exemplo, o fato de não se preocupar com a documentação, dividir o código em várias tarefas individuais que são resolvidas por pessoas diferentes simultaneamente, futuras atualizações são incrementadas, sem precisar se preocupar com revisar todo o código a cada atualização, não se preocupar tanto com os requisitos de software de maneira mais estável.

3 - Em casos que necessitam de um software que necessita mais robustez, e um tempo de resposta em tempo real, como por exemplo, o sistema de tráfego aéreo de um aeroporto, não é recomendado a utilização de softwares ágeis, pois para esses casos, é necessário um software que comunique perfeitamente com todo o sistema para que se obtenha respostas precisas com tempo de resposta baixo.

4 - Vantagens: Novas versões podem ser desenvolvidas de maneira extremamente rápida; feedback ouvido de maneira rápida pelos desenvolvedores; linguagem mais simples para os usuários; planejamento facilitado.

Desvantagens: Necessidade do cliente presente constantemente; Se um cliente chave sair do projeto, pode ocorrer uma perda de conhecimento; Não é recomendado para sistemas críticos; Trabalho com um volume muito grande de histórias pode ser muito trabalhoso.

5 - Pelo fato de possuir testes de aceitação constantemente, ou seja, os dados são testados pelo cliente sempre, o desenvolvedor consegue ir se aproximando cada vez mais das necessidades do cliente à cada novo release. No entanto, pelo fato de

depende tanto do feedback do cliente, que muitas vezes não pode ajudar tão regularmente o desenvolvedor, pode acabar atrasando ou acabar não atendendo ao pedido como ele deseja.

6 - O código é revisado em tempo real e o número de bugs pode ser reduzido, pois enquanto um programador digita o código, o outro vai analisando e revisando o código; No caso de bloqueio criativo, ou falta de idéias, como são duas pessoas ao mesmo tempo, eles podem conversar e realizar um brainstorm de ideias para desenvolver; Pelo fato de terem duas pessoas trabalhando juntas, uma observa a outra, o que ajuda a eliminação de distrações, e mais foco no trabalho; Decisões de design melhores, e outras ideias surgem naturalmente enquanto produzem o código.

7 - O scrum é um framework focado em organização e gestão do trabalho, enquanto o XP é um framework focado em engenharias e práticas técnicas de programação. No scrum, o foco é a gestão de fluxo do trabalho e produtividade da equipe, possui um ciclo de 3 - 4 semanas, possui meetings, dailys, e evita ditar como o código deve ser feito. Já o XP, foca mais na excelência técnica e qualidade do código, dura 1 a 2 semanas, e foca mais em utilizar regras técnicas.

8 - A - Vantagens:

É a estratégia que melhor se adequa às exigências de autoridades de certificação (como ANAC, FAA, EASA e a norma DO-178C). Essas entidades exigem documentação exaustiva e rastreabilidade total de ponta a ponta.

Como os requisitos são congelados precocemente, o engenheiro de suporte tem estabilidade para traduzi-los em modelos matemáticos e especificações formais sem que eles mudem o tempo todo.

Permite projetar e revisar a arquitetura do sistema como um todo antes de escrever qualquer código, reduzindo riscos de falhas estruturais graves.

Desvantagens:

É extremamente difícil, lento e custoso alterar requisitos caso seja descoberto um erro de concepção nas fases finais (como durante os testes).

Os *stakeholders* só verão o software funcionando meses ou anos após o início do projeto.

B - Vantagens:

Reduz drasticamente as ambiguidades. Os engenheiros e *stakeholders* podem ver e testar a lógica ou a interface antes da especificação formal, garantindo que estão documentando o sistema "certo".

Permite testar algoritmos complexos de controle em linguagens dinâmicas (Python/Ruby) rapidamente antes de implementá-los na linguagem final.

Desvantagens:

Exige esforço para criar um código que será obrigatoriamente jogado fora.

Stakeholders não técnicos podem ver o protótipo funcionando e não entender por que precisam esperar meses para o sistema em Java ficar pronto, gerando pressão para colocar código não-seguro (script) em produção.

Linguagens de script geralmente não lidam bem com restrições de tempo real (*Real-Time Operating Systems*) e alocação de memória essenciais para aeronaves, o que significa que o protótipo não validará o comportamento de hardware.

C - Vantagens:

O produto evolui exatamente conforme as necessidades reais, e erros de usabilidade ou de lógica superficial são corrigidos imediatamente.

É a melhor estratégia se os requisitos operacionais da aeronave ainda estiverem muito incertos e precisarem evoluir junto com o hardware.

Desvantagens:

Métodos ágeis desencorajam documentação pesada. Sem documentação rigorosa e congelada, é impossível manter a "especificação formal" atualizada a cada ciclo semanal. As agências reguladoras não certificam softwares sem rastreabilidade fixa. A arquitetura emergente (feita aos poucos) pode não suportar os rígidos requisitos de segurança, tolerância a falhas e redundância necessários para um avião.

Um piloto na equipe pode garantir que a interface seja boa, mas não tem a expertise para validar requisitos de segurança de sistemas embutidos e tolerância a falhas elétricas.

9 - 1 - Em vez de manter o mesmo usuário alocado no projeto do início ao fim, a empresa substitui esse representante por outro de perfil semelhante a cada poucos meses ou após grandes entregas (releases).

Vantagens: Garante um fluxo constante de perspectivas "frescas" e não contaminadas pela rotina da equipe de desenvolvimento.

Evita que o sistema seja construído com base nos vícios e preferências de uma única pessoa.

Desvantagens: Gera uma curva de aprendizado constante. Cada novo usuário precisará de tempo para entender como a equipe de software trabalha, os jargões básicos e o estágio atual do projeto.

Pode causar mudanças abruptas de direção no produto se o novo usuário discordar fundamentalmente das decisões tomadas pelo usuário anterior.

2 - O usuário não é alocado 100% do tempo para a equipe de desenvolvimento. Ele divide sua semana: passa alguns dias trabalhando no projeto de software e o restante do tempo executando sua função original (operacional) no negócio.

Vantagens: O usuário continua vivenciando as "dores" diárias do processo atual, mantendo total empatia pelas necessidades reais dos seus colegas.

Ele valida naturalmente as ideias do software contra a realidade do seu trabalho diário.

Desvantagens: Cria um gargalo para a equipe de desenvolvimento. Se os programadores precisarem tirar uma dúvida urgente nos dias em que o usuário está focado na operação, o trabalho pode travar.

Pode gerar sobrecarga física e mental para o usuário, que passa a ter que gerenciar "dois empregos" simultaneamente.

3 - O usuário embedded (infiltrado na equipe) atua menos como um "ditador de requisitos" e mais como um facilitador. Institui-se a regra de que suas opiniões pessoais não são suficientes; ele deve, obrigatoriamente, testar cada protótipo e nova funcionalidade com um grupo de colegas no ambiente real de trabalho antes de aprovar a entrega.

Vantagens: Protege a equipe do risco de desenvolver um sistema altamente enviesado, garantindo que o software atenda à média dos usuários, e não apenas ao representante.

O usuário representante ganha força política, pois suas decisões passam a ser respaldadas pelo feedback de seus pares.

Desvantagens: Torna o processo de tomada de decisão muito mais lento, reduzindo a agilidade da equipe de desenvolvimento.

Exige que o representante tenha excelentes habilidades de comunicação e negociação para lidar com opiniões conflitantes entre os próprios colegas de operação.

10 - No trabalho remoto, toda interação precisa ser intencional, e aquele aprendizado através de conversas, perguntas (um conhecimento adquirido pela comunicação) acaba sendo perdido; A programação em par funcionar online exige uma estrutura bem robusta, com latência baixa; Planejamento visual perde um pouco com a falta de quadros, cartões de histórias; Distanciamento de cliente e do usuário (fisicamente).

Para evitar esses problemas, são necessárias ferramentas de pareamento síncrono, implementação de um melhor sistema de salas virtuais, uma melhor gestão visual e de comunicação e uma flexibilização melhor dos horários de trabalho.